`vector<int>`与`int arr[]`互相转换的完整方法解析

📌 `vector<int>` 与 `int arr[]` 互相转换的完整方法

核心原则：`vector` 是C++封装的动态数组（带内存管理），`int arr[]` 是C风格静态数组（退化为指针），二者转换的关键是**利用 `vector` 的底层数组特性 + 明确数组长度**（避免野指针/越界）。

一、C风格数组 `int arr[]` → `vector<int>`（最常用，推荐）

方法1：直接用 `vector` 构造函数（最简）

利用 `vector` 支持「首地址+尾地址」的构造方式，一步完成转换，效率最高。

Code block

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    // 1. 定义C风格数组并指定长度
    int arr[] = {1, 2, 3, 4, 5};
    // 计算数组长度：总字节数 / 单个元素字节数（必须！否则不知道数组边界）
    int arr_len = sizeof(arr) / sizeof(arr[0]);

    // 2. 直接构造vector（核心：传入首地址arr + 尾地址arr+arr_len）
    vector<int> vec(arr, arr + arr_len);

    // 验证结果
    for (int num : vec) {
        cout << num << " "; // 输出：1 2 3 4 5
    }
    return 0;
}
```

方法2：循环 `push_back`（适合需要逐元素处理的场景）

如果转换时要对元素做修改（比如过滤、计算），用这种方式更灵活：

Code block

```
#include <vector>
#include <iostream>
using namespace std;

int main() {
    int arr[] = {1, 2, 3, 4, 5};
    int arr_len = sizeof(arr) / sizeof(arr[0]);

    vector<int> vec;
    // 预留空间（可选，提升性能，避免vector频繁扩容）
    vec.reserve(arr_len);
    for (int i = 0; i < arr_len; i++) {
        vec.push_back(arr[i] * 2); // 示例：转换时将元素翻倍
    }

    for (int num : vec) {
        cout << num << " "; // 输出：2 4 6 8 10
    }
    return 0;
}
```

二、`vector<int>` → C风格数组 `int arr[]`（注意内存安全）

`vector` 无法直接“赋值”给 `int arr[]`（因为 `arr[]` 是静态数组，长度固定），核心思路是**拷贝 `vector` 的底层数据到C数组，或获取 `vector` 底层数组的指针**。

方法1：拷贝数据到已定义的C数组（最安全）

先定义长度匹配的C数组，再用 `memcpy` 或循环拷贝 `vector` 的元素：

Code block

```
#include <vector>
#include <cstring> // memcpy需要的头文件
#include <iostream>
using namespace std;

int main() {
    // 1. 定义并初始化vector
    vector<int> vec = {10, 20, 30, 40, 50};
    int vec_len = vec.size();

    // 2. 定义长度匹配的C数组（必须保证长度≥vector.size()，否则越界）
    int arr[5];
```

```
    // 方法A：memcpy批量拷贝（高效）
    memcpy(arr, vec.data(), vec_len * sizeof(int));
    // 方法B：循环拷贝（适合需要逐元素处理）
    // for (int i = 0; i < vec_len; i++) {
    //     arr[i] = vec[i];
    // }

    // 验证结果
    for (int i = 0; i < vec_len; i++) {
        cout << arr[i] << " "; // 输出：10 20 30 40 50
    }
    return 0;
}
```

方法2：获取 `vector` 底层数组的指针（临时使用）

`vector` 的底层是连续内存，可用 `data()`（C++11+）或 `&vec[0]` 获取首地址，直接当作C数组使用（**注意：不能修改vector的大小，否则指针失效！**）：

Code block

```
#include <vector>
#include <iostream>
using namespace std;

// 模拟C风格函数：接收int数组指针+长度
void print_c_array(int* arr, int len) {
    for (int i = 0; i < len; i++) {
        cout << arr[i] << " ";
    }
}

int main() {
    vector<int> vec = {1, 2, 3, 4, 5};

    // 核心：vec.data() 返回vector底层数组的首地址（等价于 &vec[0]）
    int* c_arr_ptr = vec.data();
    int vec_len = vec.size();

    // 传入C风格函数使用
    print_c_array(c_arr_ptr, vec_len); // 输出：1 2 3 4 5

    // ❌ 危险：如果修改vector大小，指针会失效！
    // vec.push_back(6); // 可能触发扩容，c_arr_ptr变成野指针
    return 0;
}
```

⚠ 关键注意事项

1. **长度必须明确**：C数组退化为指针后丢失长度信息，转换时必须手动传递/计算长度，否则会越界。
2. **内存安全**：
   - 从 `vector` 获取的指针，不能在 `vector` 扩容/析构后使用（会变成野指针）；
   - 拷贝到C数组时，必须保证C数组长度 ≥ `vector.size()`。
3. **兼容性**：`vec.data()` 是C++11新增，旧编译器可用 `&vec[0]`（但 `vec` 不能为空，否则越界）。

总结

1. **C数组 → vector**：优先用 `vector(arr, arr+len)` 构造函数，简洁高效；需要处理元素时用 `push_back` 循环。
2. **vector → C数组**：优先用 `memcpy` /循环拷贝到已定义的C数组（安全）；临时使用可直接取 `vec.data()` 指针（禁止修改vector大小）。
3. 核心：二者转换的本质是**数据拷贝**（或临时借用底层指针），需始终关注数组长度和内存有效性。

（注：文档部分内容可能由 AI 生成）